Аллокация памяти

Вспомним…

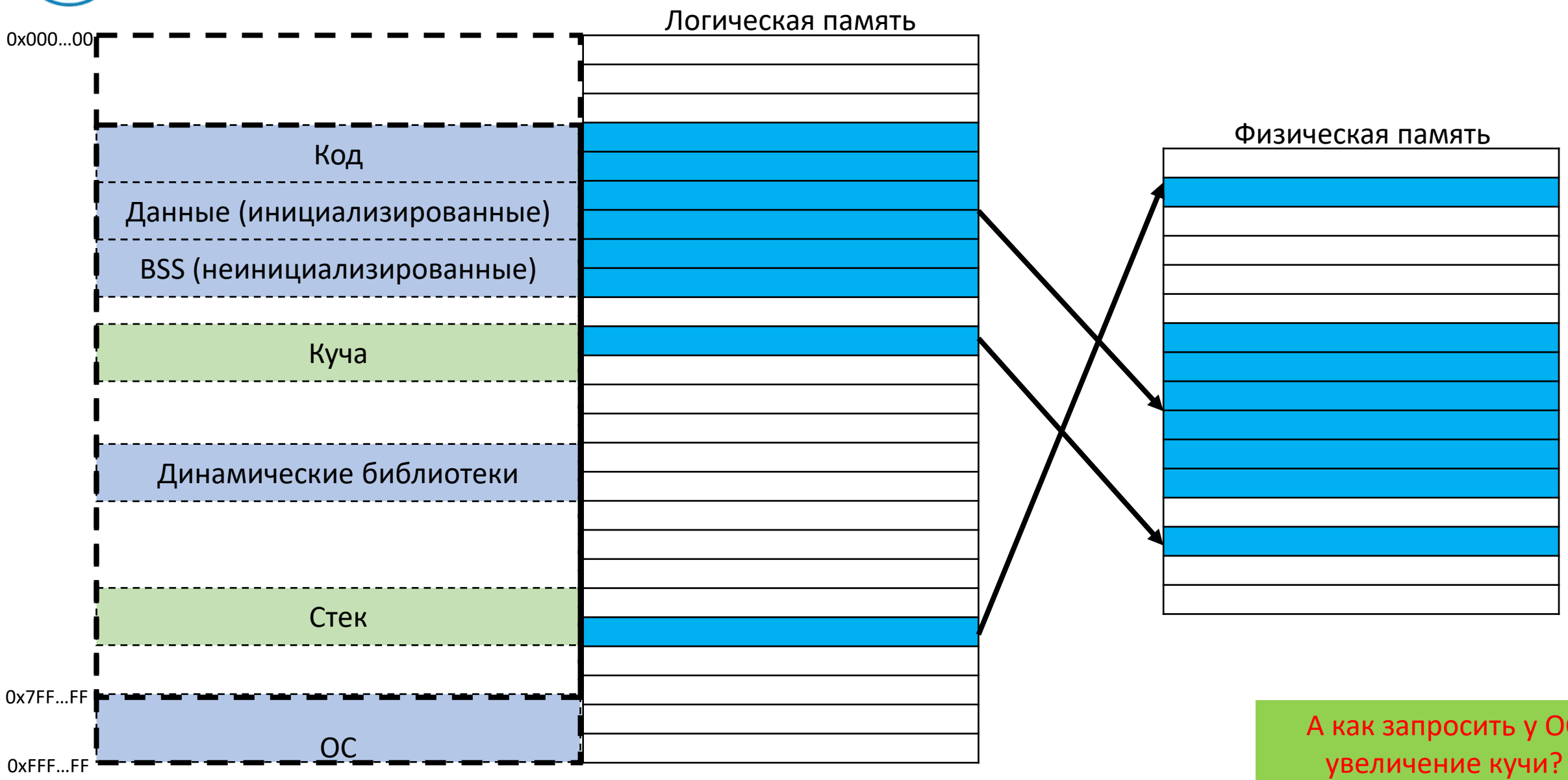
Какой размер указателя?

В современных ПК - 64 бита

2^{64}бит =18 446 744 073 709 551 616 бит ~ 17 179 869 184 ГБ.

Секции программы

Страничная организация памяти



А как запросить у ОС увеличение кучи?

Аллокация памяти

Системные вызовы ОС *Linux* для работы с памятью:

brk()/sbrk() — используется для изменения объема памяти, выделенной процессу.

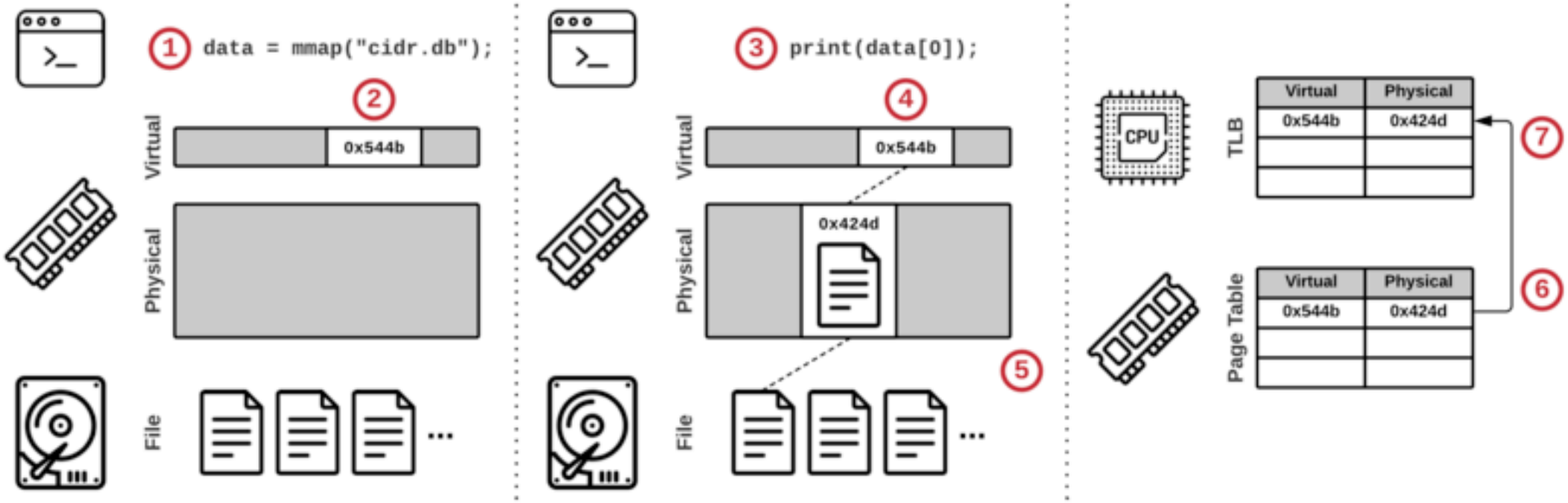
mmap() – позволяет отображать память из любого места процесса

Task 052

Аллокация памяти

С помощью системного вызова ***sbrk()*** возможно изменять положение **Program break**

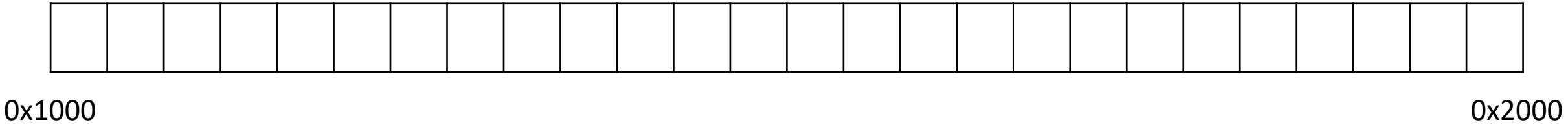
mmap



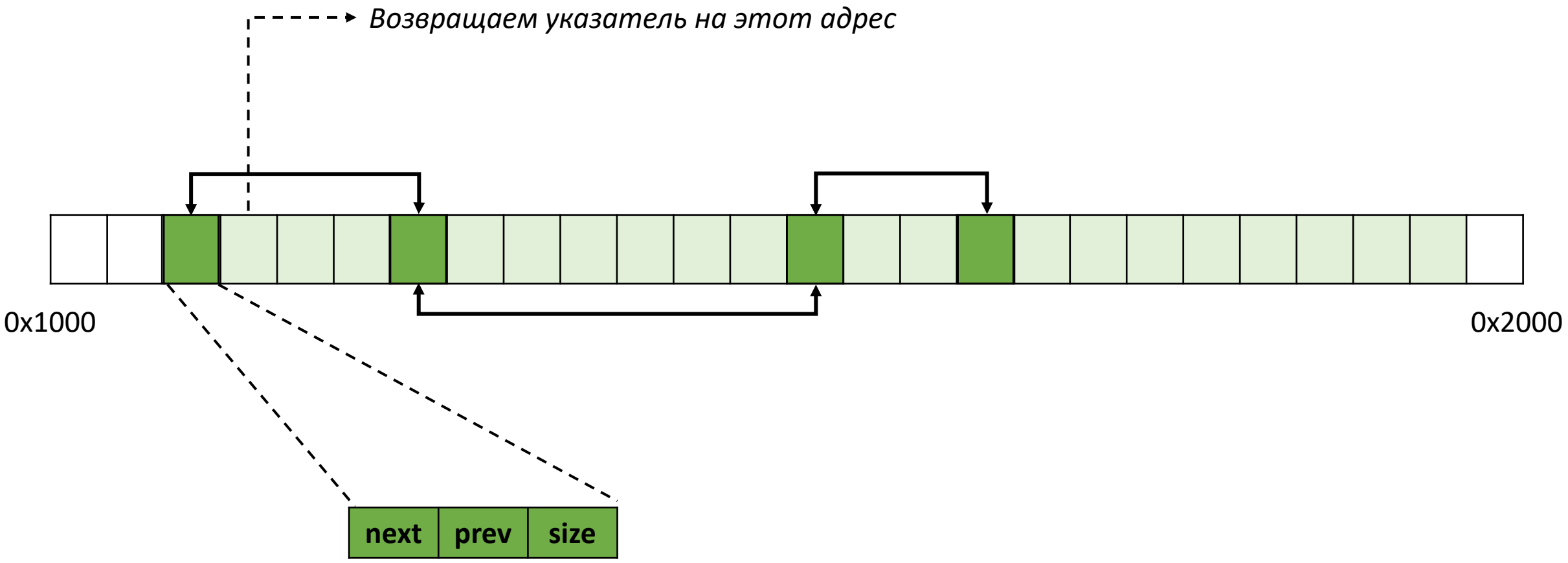
mmap в Linux — это системный вызов и библиотечная функция, позволяющая отображать файлы или другие объекты в память. Он предоставляет способ работы с файлами как с массивами в памяти, что может быть быстрее и удобнее, чем использование стандартных функций read() и write()

Алгоритмы аллокации памяти

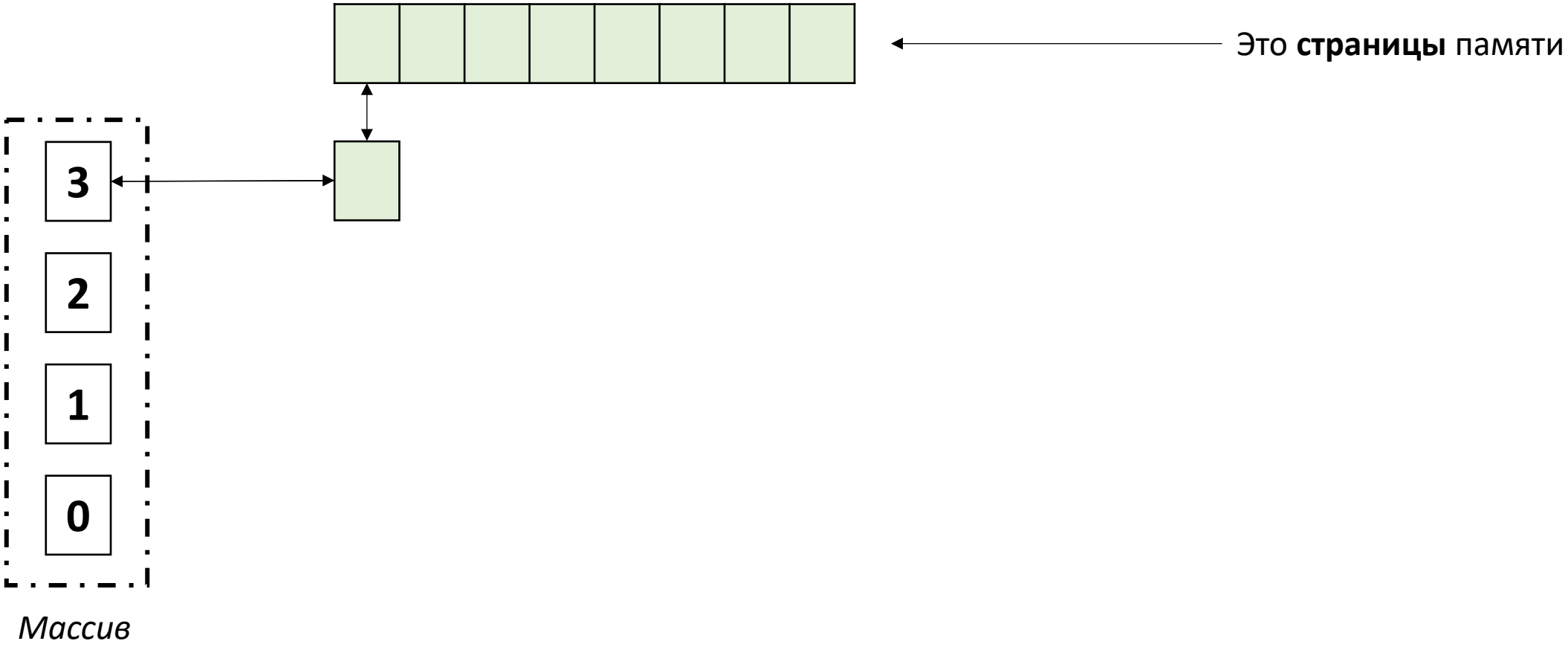
Есть непрерывный участок логической памяти.
Как правильно и эффективно выделять участки?



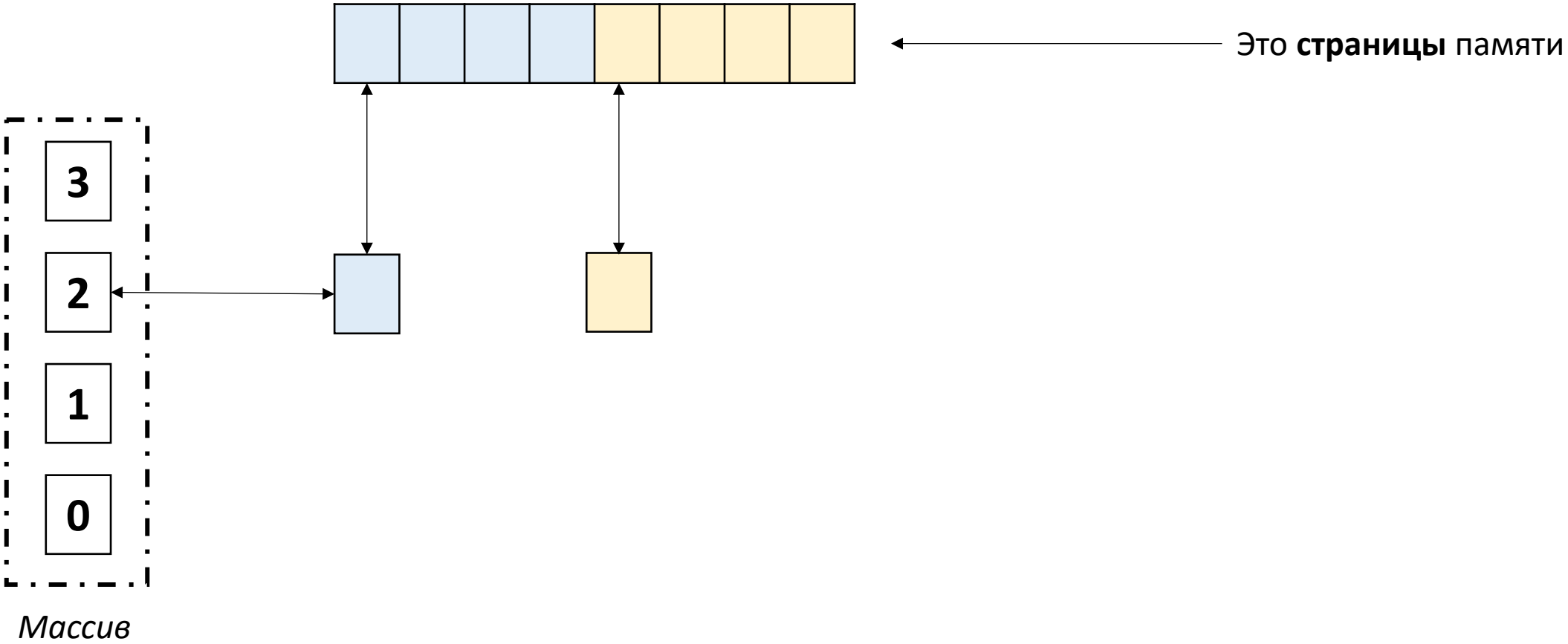
Первый подход – «в лоб»



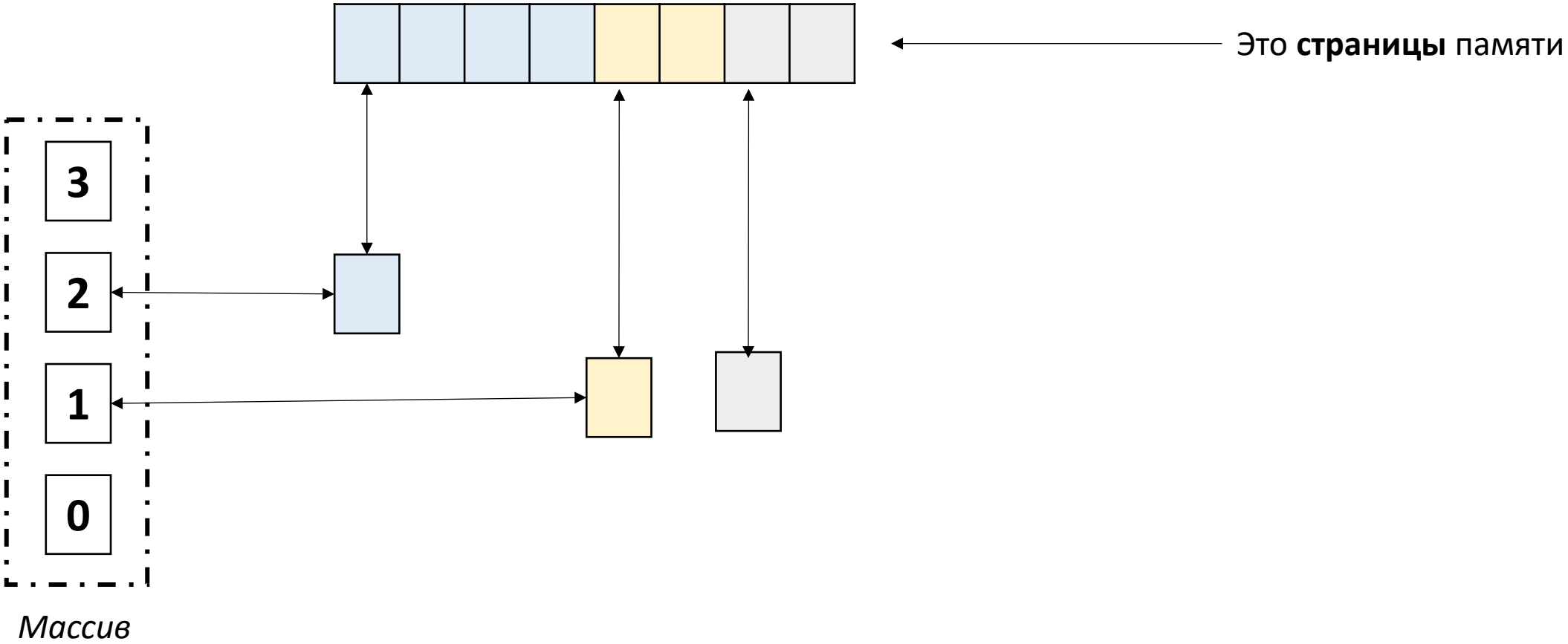
Buddy аллокатор



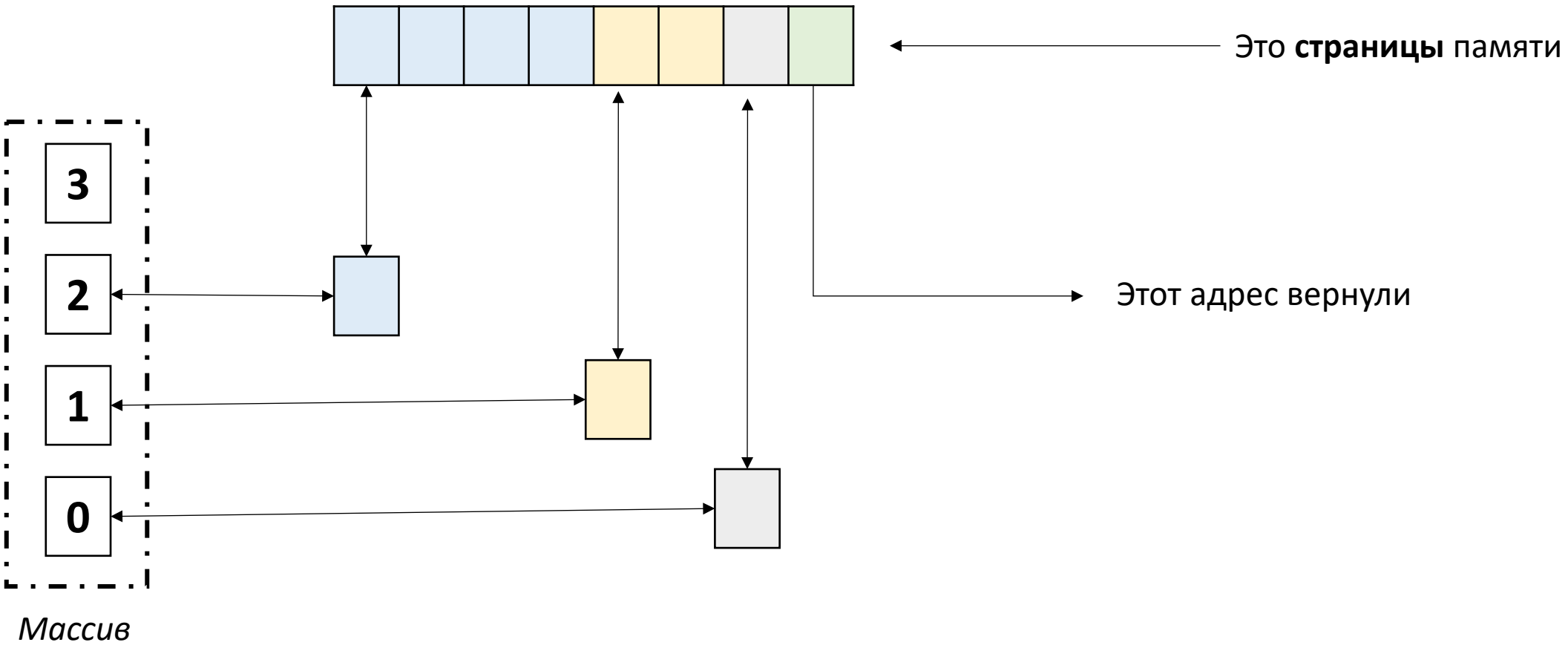
Buddy аллокатор



Buddy аллокатор

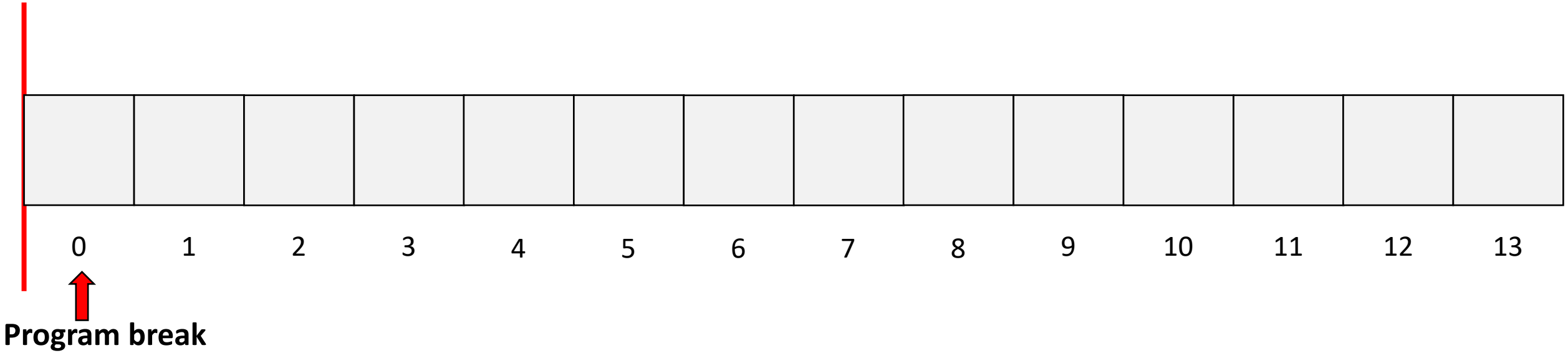


Buddy аллокатор



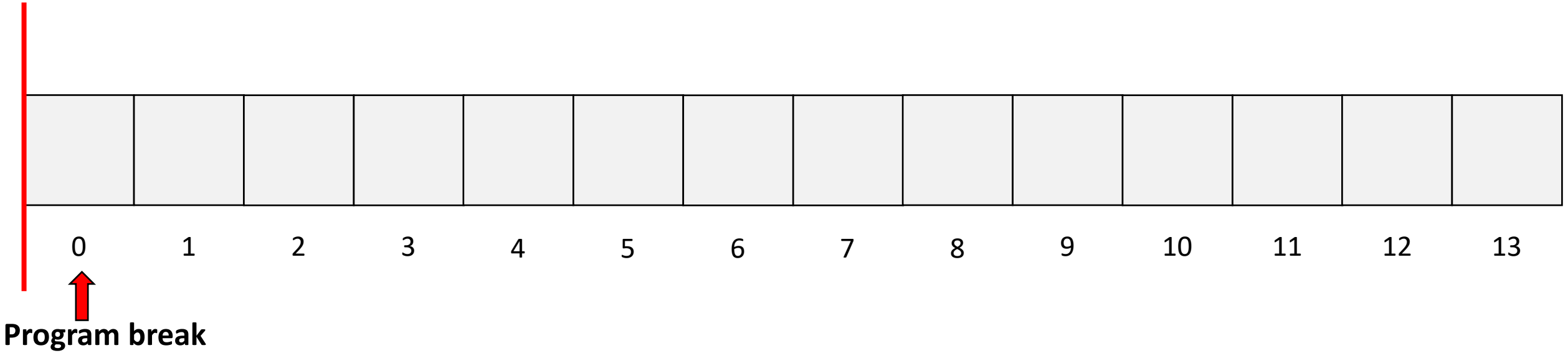
Аллокация памяти

```
int* x = (int*)malloc(N);
```



Аллокация памяти

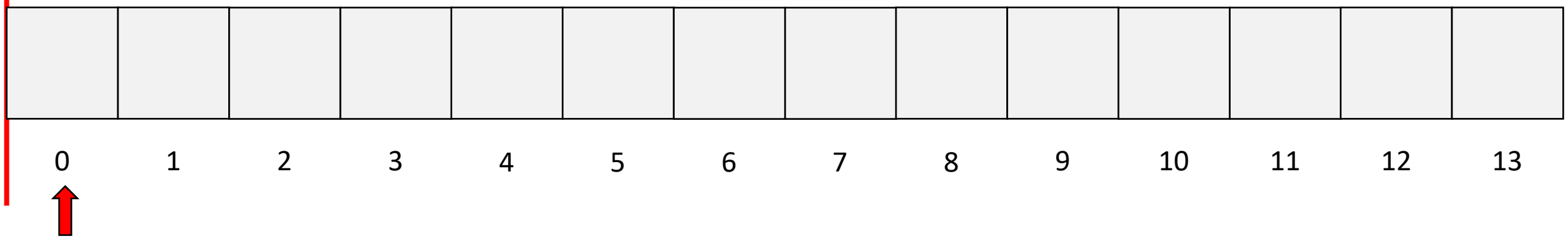
```
int* x = (int*)malloc(N);
```




```
struct mem_control_block {
        int is_available;
        int size;
};
```

Аллокация памяти

```
int* x = (int*)malloc(3);
```



program break

current_location

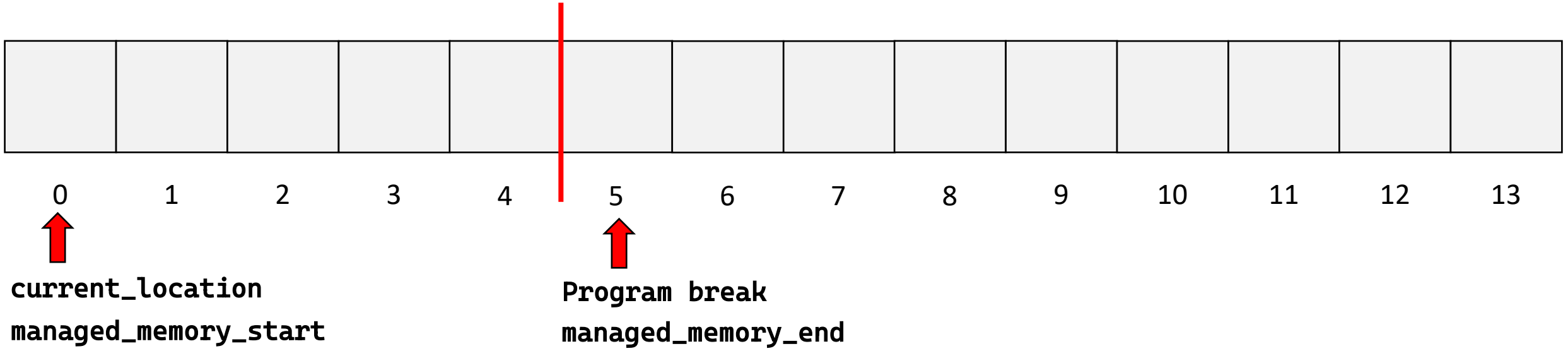
managed_memory_start

managed_memory_end

```
managed_memory_end = sbrk(0);                          = 0
managed_memory_start = managed_memory_end;             = 0
current_location = managed_memory_start;               = 0
numbytes = numbytes + sizeof(struct mem_control_block);  = 3 + 2 = 5
```

Аллокация памяти

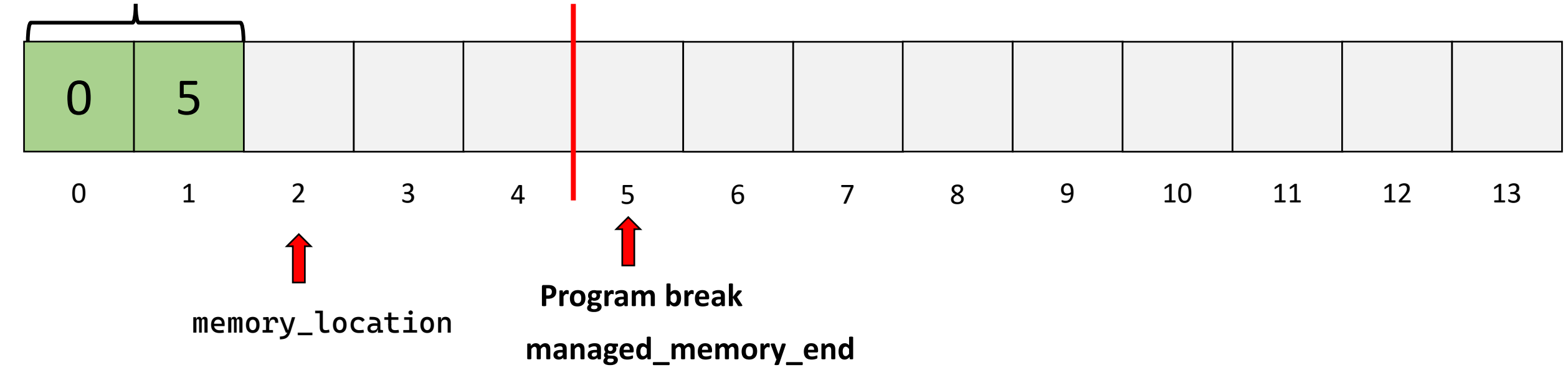
```
int* x = (int*)malloc(3);
```




```
sbrk(numbytes);                                          sbrk(5) = 5
memory_location = managed_memory_end;                    = 0
managed_memory_end = managed_memory_end + numbytes;      = 0 + 5
current_location_mcb = memory_location;                  = 0
current_location_mcb->is_available = 0;
current_location_mcb->size = numbytes;                   = 5
```

Аллокация памяти

```
int* x = (int*)malloc(3);
```

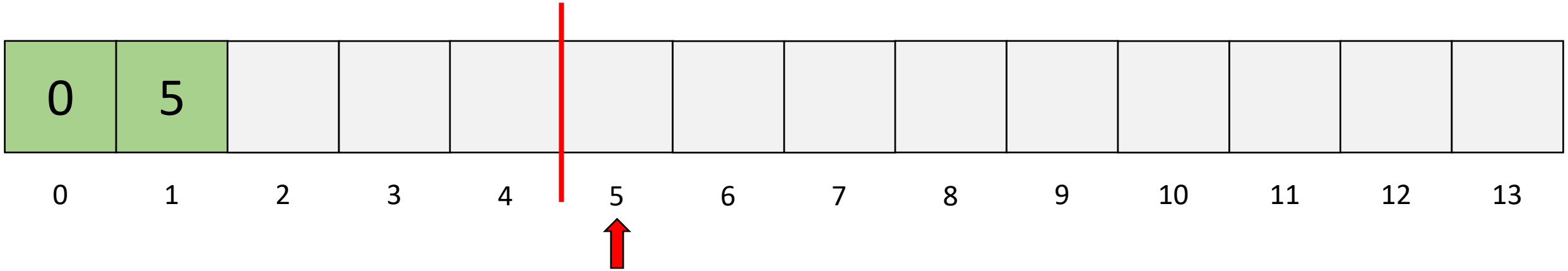



```
memory_location = memory_location + sizeof(struct mem_control_block); = 0 + 2 = 2;

return memory_location;
```

Аллокация памяти

```
int* y = (int*)malloc(5);
```



current_location

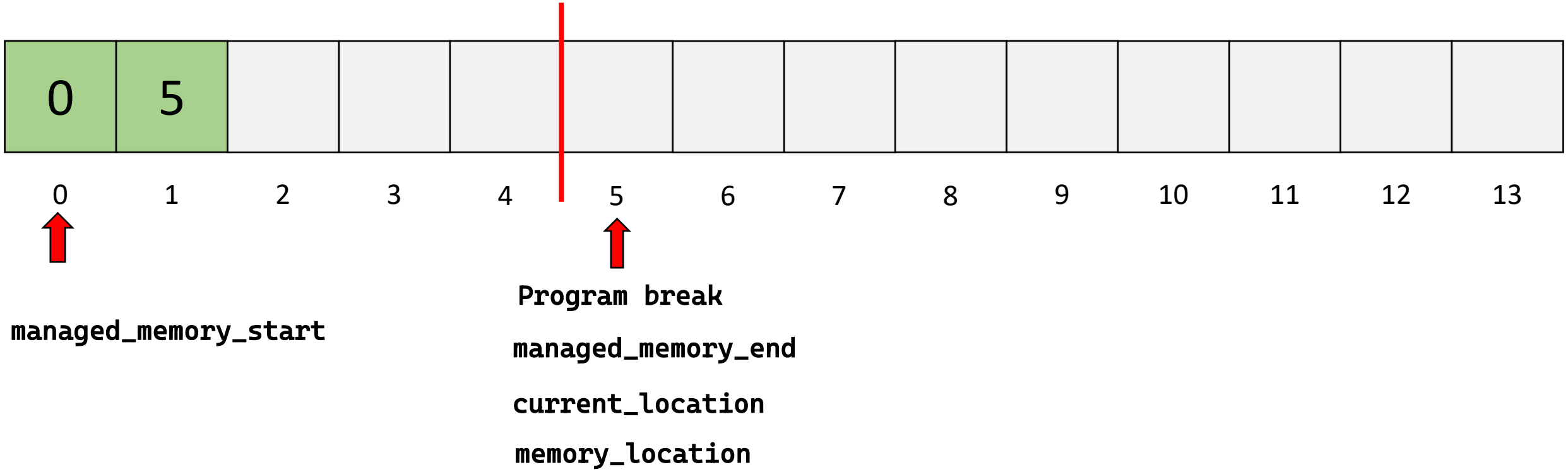
managed_memory_end

```
while (current_location != managed_memory_end){
        if (current_location_mcb->is_available) {//выделяем память}
        current_location = current_location + current_location_mcb->size;
}
```

Ячейка занята, поэтому двигаемся дальше `current_location = 0 + 5 = 5`

Аллокация памяти

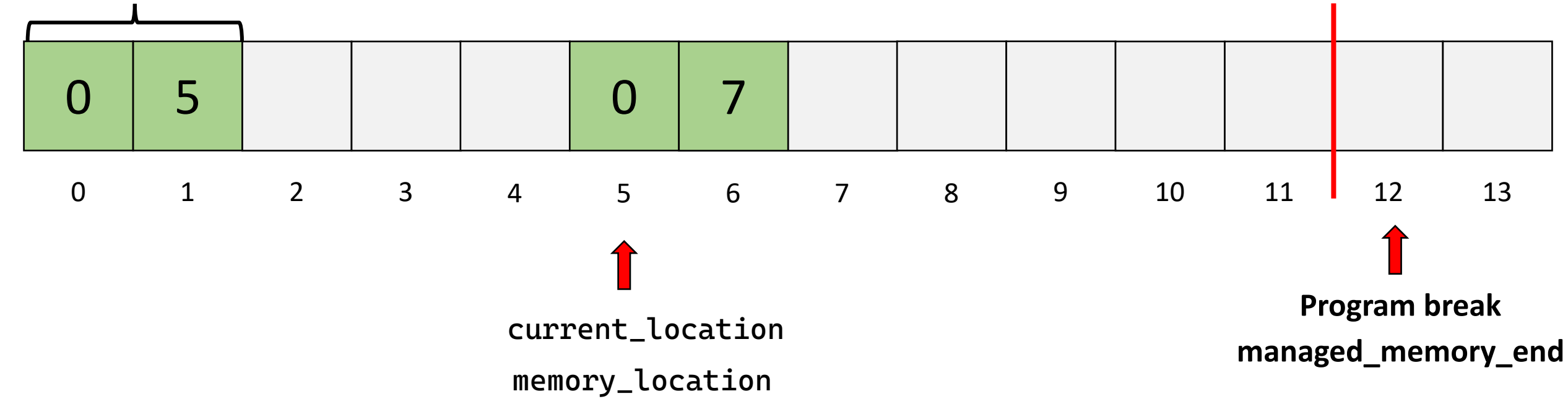
```
int* y = (int*)malloc(5);
```



Цикл закончился -> снова необходимо просить памяти

Аллокация памяти

```
int* y = (int*)malloc(5);
```




```
memory_location = memory_location + sizeof(struct mem_control_block);
memory_location = 5 + 2 = 7;

return memory_location;
```

Аллокация памяти

```
free(x);
```

Адрес начала выделенной памяти

```
mcb = x - sizeof(struct mem_control_block);
mcb->is_available = 1;
```

Аллокация памяти

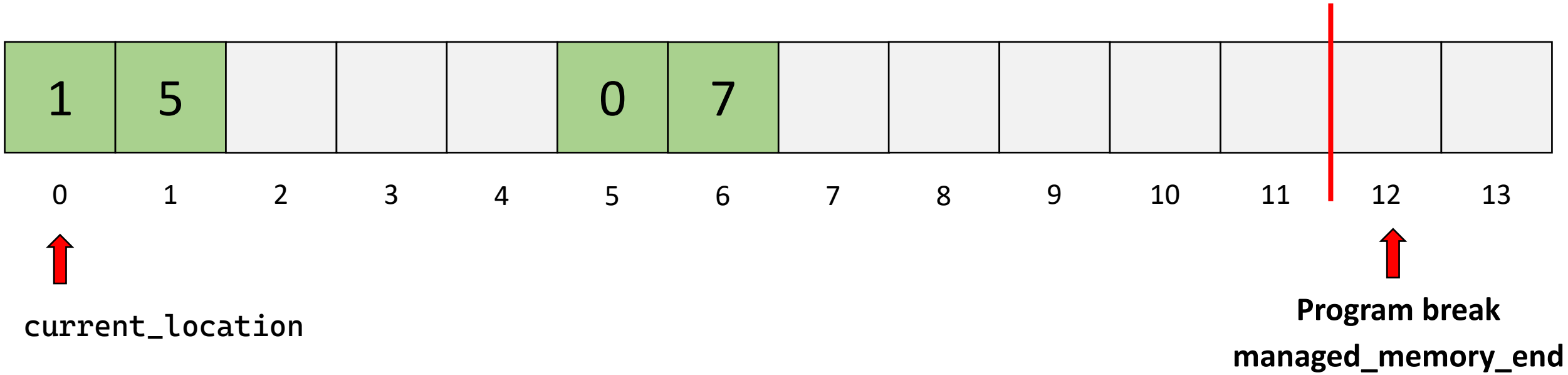
```
free(x);
```

Адрес начала выделенной памяти

```
mcb = x - sizeof(struct mem_control_block);
mcb->is_available = 1;
```

Аллокация памяти

```
int* y = (int*)malloc(2);
```

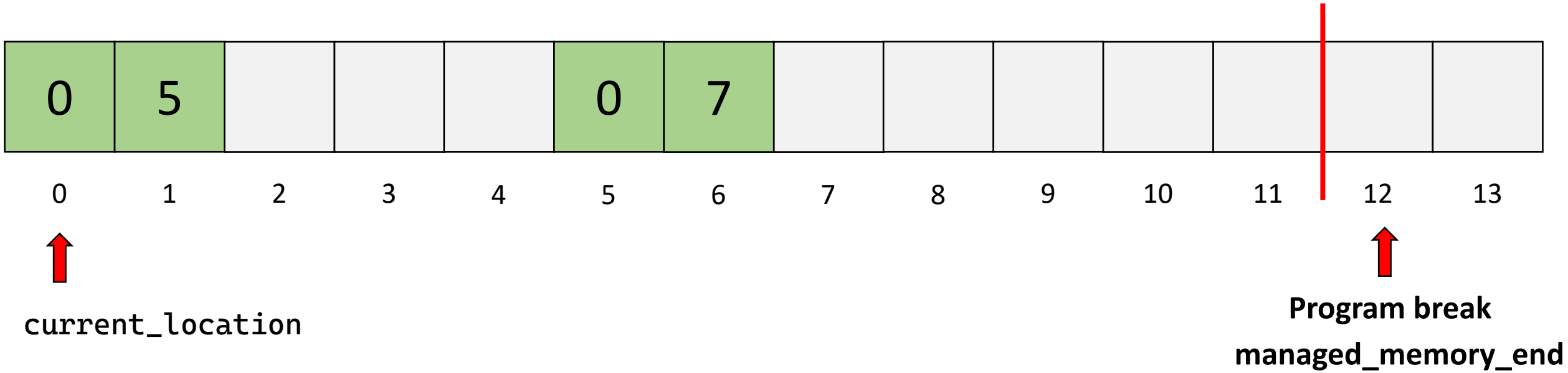



```
while (current_location != managed_memory_end){
        if (current_location_mcb->is_available) {//выделяем память}
        current_location = current_location + current_location_mcb->size;
}
```

Ячейка свободна, поэтому используем её

Аллокация памяти

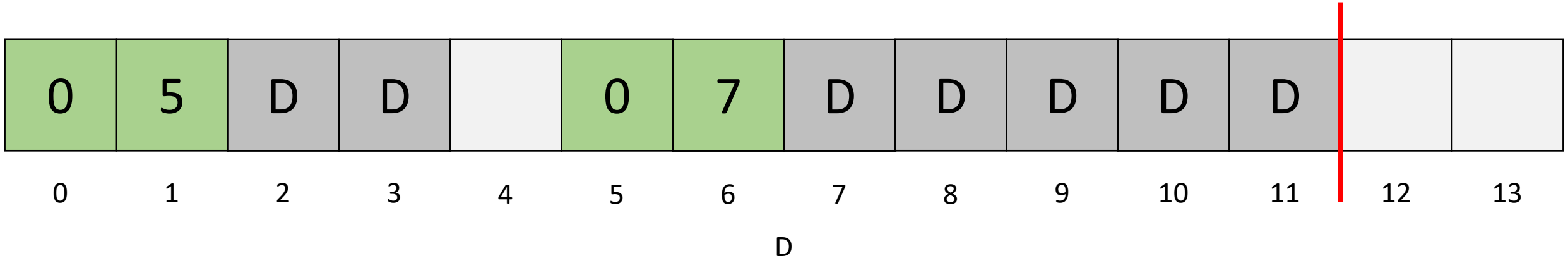
```
int* y = (int*)malloc(2);
```




```
while (current_location != managed_memory_end){
        if (current_location_mcb->is_available) {//выделяем память}
        current_location = current_location + current_location_mcb->size;
}
```

Ячейка свободна, поэтому используем её

Недостатки реализации



1. Неэффективное использование памяти:
    1.1. Пустые ячейки (например, 4)
    1.2. Положение структур невозможно изменить
2. Медленная работа